



**UNSW**  
SYDNEY

# COMP4141

Theory of Computation

Lecture 2: Regular Languages, Finite Automata

# Outline

## Motivation

Deterministic Finite Automata

Non-deterministic Finite Automata

NFAs vs. DFAs

$\epsilon$ -NFA

End Matter

## Regular languages

The next few lectures will be about *regular languages*.

Regular languages can be infinite, and can be described in several ways: by deterministic or non-deterministic finite automata, or by regular expressions. Regular languages are of great practical as well as theoretical importance.

- Widely used in applications (compilers, pattern matching, specification and formal verification of systems, TCP/IP, game AI).
- Constructions from automata theory are used in these applications.
- Robust — many representations are equivalent; they are closed under many operations.
- Basis for more sophisticated representations (automata on infinite strings, tree automata, timed automata).
- Answers to many questions about finite automata can be computed. (membership, emptiness, universality, subset, etc.)

# A Model of Computation

This lecture will be about *State machines*.

State machines model step-by-step processes:

- Set of “states”, possibly including a designated “start state”
- For each state, a set of actions detailing how to move (transition) to other states

## Example

The execution of a program:

- States: “Snapshots” of the computer at a given instant: e.g. variable values, function stack
- Transitions: defined by the program

# A Model of Computation

This lecture will be about *State machines*.

State machines model step-by-step processes:

- Set of “states”, possibly including a designated “start state”
- For each state, a set of actions detailing how to move (transition) to other states

## Example

A chess solving engine

- States: Board positions
- Transitions: Legal moves

# A Model of Computation

This lecture will be about *State machines*.

State machines model step-by-step processes:

- Set of “states”, possibly including a designated “start state”
- For each state, a set of actions detailing how to move (transition) to other states

## Example

“Stateful” communication protocols: e.g. SMTP

- States: Stages of communication
- Transitions: Determined by commands given (e.g. HELO, DATA, etc)

# A Model of Computation

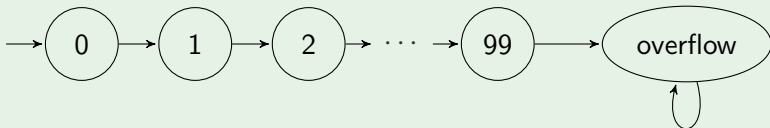
This lecture will be about *State machines*.

State machines model step-by-step processes:

- Set of “states”, possibly including a designated “start state”
- For each state, a set of actions detailing how to move (transition) to other states

## Example

A bounded counter that counts from 0 to 99 and overflows at 100:



# A Model of Computation

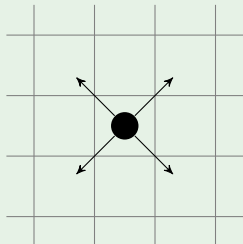
This lecture will be about *State machines*.

State machines model step-by-step processes:

- Set of “states”, possibly including a designated “start state”
- For each state, a set of actions detailing how to move (transition) to other states

## Example

A robot that moves diagonally



States: Locations

Transitions: Moves



# A Model of Computation

This lecture will be about *State machines*.

State machines model step-by-step processes:

- Set of “states”, possibly including a designated “start state”
- For each state, a set of actions detailing how to move (transition) to other states

## Example

Die Hard jug problem: Given jugs of 3L and 5L, measure out exactly 4L.

- States: Defined by amount of water in each jug
- Start state: No water in both jugs
- Transitions: Pouring water (in, out, jug-to-jug)

# Outline

Motivation

**Deterministic Finite Automata**

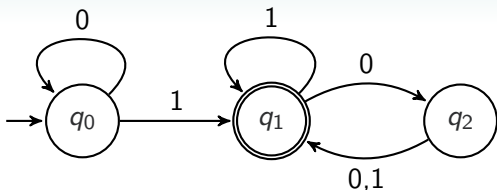
Non-deterministic Finite Automata

NFAs vs. DFAs

$\epsilon$ -NFA

End Matter

# Deterministic Finite Automata



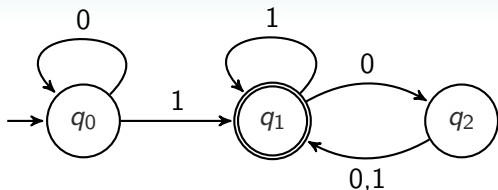
A finite automaton is a mathematical model of a machine that reads strings and says “yes” or “no” for each string.

A **deterministic finite automaton** (*DFA*) defines a potentially infinite language: the set of strings for which it says “yes”.

DFAs represent “computation with finite memory”

DFAs form the backbone of most computational models

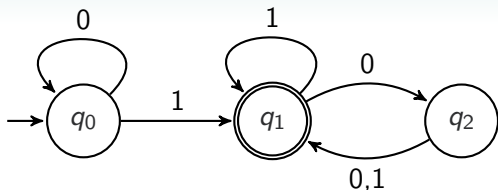
# Deterministic Finite Automata



Formally, a **deterministic finite automaton (DFA)** is a tuple  $(Q, \Sigma, \delta, q_0, F)$  where

- $Q$  is a finite set of states
- $\Sigma$  is the input alphabet
- $\delta : Q \times \Sigma \rightarrow Q$  is the transition function
- $q_0 \in Q$  is the start state
- $F \subseteq Q$  is the set of final/accepting states

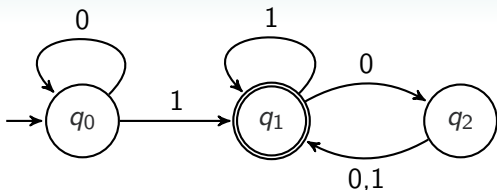
# Deterministic Finite Automata



Formally, a **deterministic finite automaton (DFA)** is a tuple  $(Q, \Sigma, \delta, q_0, F)$  where

- $Q$  is a finite set of states:  $Q = \{q_0, q_1, q_2\}$
- $\Sigma$  is the input alphabet
- $\delta : Q \times \Sigma \rightarrow Q$  is the transition function
- $q_0 \in Q$  is the start state
- $F \subseteq Q$  is the set of final/accepting states

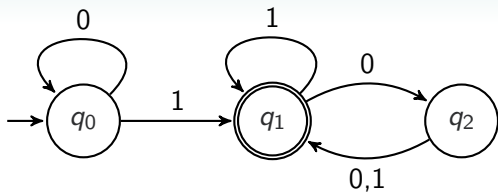
# Deterministic Finite Automata



Formally, a **deterministic finite automaton (DFA)** is a tuple  $(Q, \Sigma, \delta, q_0, F)$  where

- $Q$  is a finite set of states:  $Q = \{q_0, q_1, q_2\}$
- $\Sigma$  is the input alphabet:  $\Sigma = \{0, 1\}$
- $\delta : Q \times \Sigma \rightarrow Q$  is the transition function
- $q_0 \in Q$  is the start state
- $F \subseteq Q$  is the set of final/accepting states

# Deterministic Finite Automata



$$\delta(q_0, 0) = q_0$$

$$\delta(q_0, 1) = q_1$$

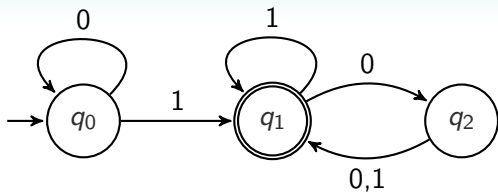
$$\delta(q_1, 0) = q_2$$

$$\delta(q_1, 1) = q_1$$

$$\delta(q_2, 0) = q_1$$

$$\delta(q_2, 1) = q_1$$

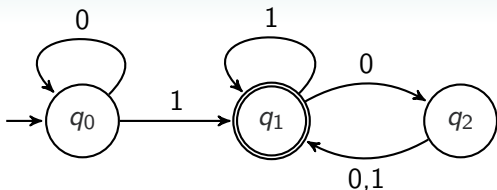
# Deterministic Finite Automata



| $\delta$ | 0     | 1     |
|----------|-------|-------|
| $q_0$    | $q_0$ | $q_1$ |
| $q_1$    | $q_2$ | $q_1$ |
| $q_2$    | $q_1$ | $q_1$ |



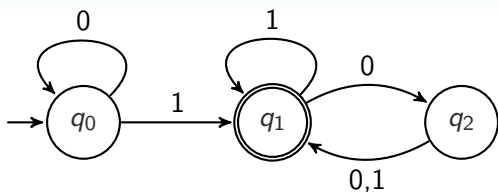
# Deterministic Finite Automata



Formally, a **deterministic finite automaton (DFA)** is a tuple  $(Q, \Sigma, \delta, q_0, F)$  where

- $Q$  is a finite set of states:  $Q = \{q_0, q_1, q_2\}$
- $\Sigma$  is the input alphabet:  $\Sigma = \{0, 1\}$
- $\delta : Q \times \Sigma \rightarrow Q$  is the transition function
- $q_0 \in Q$  is the start state
- $F \subseteq Q$  is the set of final/accepting states:  $F = \{q_1\}$

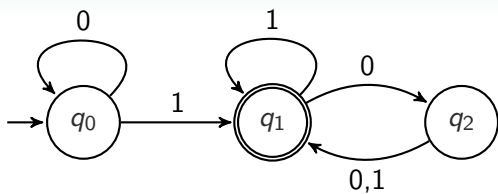
## Language of a DFA



A DFA accepts a sequence of symbols from  $\Sigma$  – i.e. elements of  $\Sigma^*$

Informally: A word defines a run in the DFA and the word is accepted if the run ends in a final state.

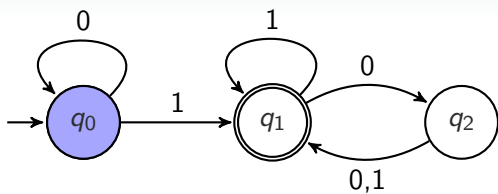
## Language of a DFA



$w$ : 1001

A DFA accepts a sequence of symbols from  $\Sigma$  – i.e. elements of  $\Sigma^*$

## Language of a DFA

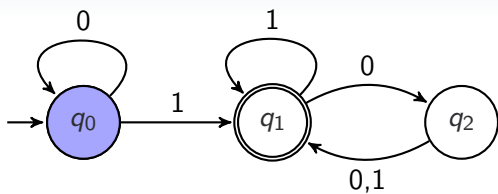


$w$ : 1001

A DFA accepts a sequence of symbols from  $\Sigma$  – i.e. elements of  $\Sigma^*$

- Start in state  $q_0$

## Language of a DFA

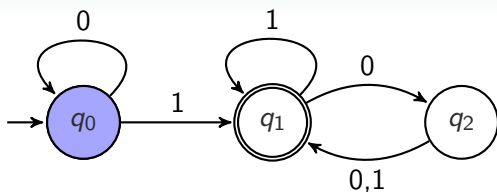


$w$ : 1001

A DFA accepts a sequence of symbols from  $\Sigma$  – i.e. elements of  $\Sigma^*$

- Start in state  $q_0$
- Take the first symbol of  $w$

## Language of a DFA

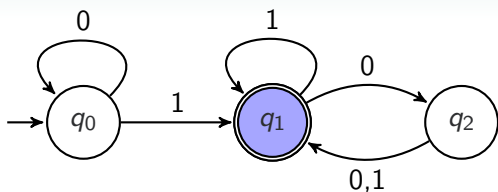


$w$ : 1001

A DFA accepts a sequence of symbols from  $\Sigma$  – i.e. elements of  $\Sigma^*$

- Start in state  $q_0$
- Take the first symbol of  $w$
- Repeat the following until there are no symbols left:
  - Based on the current state and current input symbol, transition to the appropriate state determined by  $\delta$

## Language of a DFA

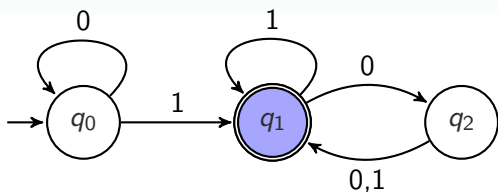


$w$ : 1001

A DFA accepts a sequence of symbols from  $\Sigma$  – i.e. elements of  $\Sigma^*$

- Start in state  $q_0$
- Take the first symbol of  $w$
- Repeat the following until there are no symbols left:
  - Based on the current state and current input symbol, transition to the appropriate state determined by  $\delta$

## Language of a DFA



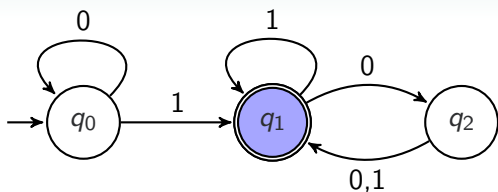
$w$ : 1001

A DFA accepts a sequence of symbols from  $\Sigma$  – i.e. elements of  $\Sigma^*$

- Start in state  $q_0$
- Take the first symbol of  $w$
- Repeat the following until there are no symbols left:
  - Based on the current state and current input symbol, transition to the appropriate state determined by  $\delta$
  - Move to the next symbol in  $w$



## Language of a DFA

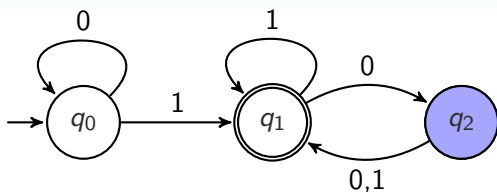


$w$ : 1001

A DFA accepts a sequence of symbols from  $\Sigma$  – i.e. elements of  $\Sigma^*$

- Start in state  $q_0$
- Take the first symbol of  $w$
- Repeat the following until there are no symbols left:
  - Based on the current state and current input symbol, transition to the appropriate state determined by  $\delta$
  - Move to the next symbol in  $w$

## Language of a DFA

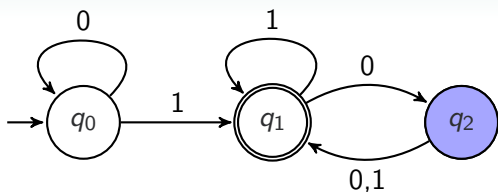


$w$ : 1001

A DFA accepts a sequence of symbols from  $\Sigma$  – i.e. elements of  $\Sigma^*$

- Start in state  $q_0$
- Take the first symbol of  $w$
- Repeat the following until there are no symbols left:
  - Based on the current state and current input symbol, transition to the appropriate state determined by  $\delta$
  - Move to the next symbol in  $w$

## Language of a DFA

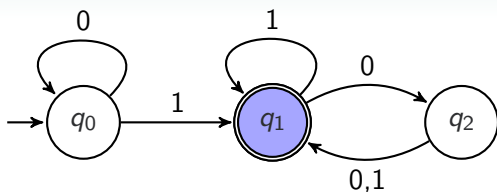


$w$ : 1001

A DFA accepts a sequence of symbols from  $\Sigma$  – i.e. elements of  $\Sigma^*$

- Start in state  $q_0$
- Take the first symbol of  $w$
- Repeat the following until there are no symbols left:
  - Based on the current state and current input symbol, transition to the appropriate state determined by  $\delta$
  - Move to the next symbol in  $w$

## Language of a DFA

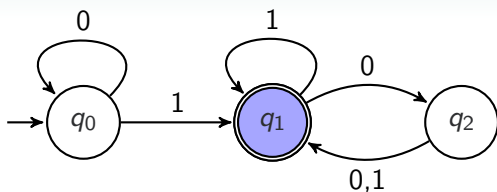


$w$ : 1001

A DFA accepts a sequence of symbols from  $\Sigma$  – i.e. elements of  $\Sigma^*$

- Start in state  $q_0$
- Take the first symbol of  $w$
- Repeat the following until there are no symbols left:
  - Based on the current state and current input symbol, transition to the appropriate state determined by  $\delta$
  - Move to the next symbol in  $w$

## Language of a DFA

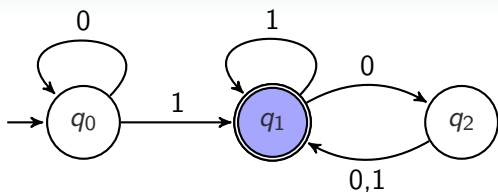


$w$ : 100**1**

A DFA accepts a sequence of symbols from  $\Sigma$  – i.e. elements of  $\Sigma^*$

- Start in state  $q_0$
- Take the first symbol of  $w$
- Repeat the following until there are no symbols left:
  - Based on the current state and current input symbol, transition to the appropriate state determined by  $\delta$
  - Move to the next symbol in  $w$

## Language of a DFA

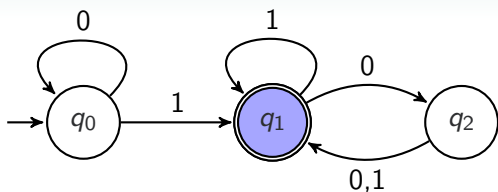


$w$ : 1001

A DFA accepts a sequence of symbols from  $\Sigma$  – i.e. elements of  $\Sigma^*$

- Start in state  $q_0$
- Take the first symbol of  $w$
- Repeat the following until there are no symbols left:
  - Based on the current state and current input symbol, transition to the appropriate state determined by  $\delta$
  - Move to the next symbol in  $w$

## Language of a DFA

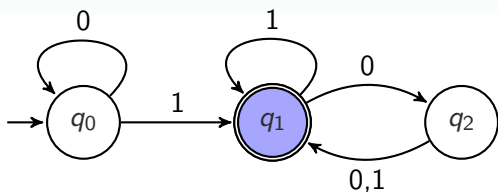


$w$ : 1001

A DFA accepts a sequence of symbols from  $\Sigma$  – i.e. elements of  $\Sigma^*$

- Start in state  $q_0$
- Take the first symbol of  $w$
- Repeat the following until there are no symbols left:
  - Based on the current state and current input symbol, transition to the appropriate state determined by  $\delta$
  - Move to the next symbol in  $w$
- Accept if the process ends in a final state, otherwise reject.

## Language of a DFA



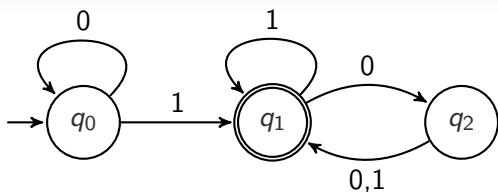
$w$ : 1001 ✓

A DFA accepts a sequence of symbols from  $\Sigma$  – i.e. elements of  $\Sigma^*$

- Start in state  $q_0$
- Take the first symbol of  $w$
- Repeat the following until there are no symbols left:
  - Based on the current state and current input symbol, transition to the appropriate state determined by  $\delta$
  - Move to the next symbol in  $w$
- Accept if the process ends in a final state, otherwise reject.

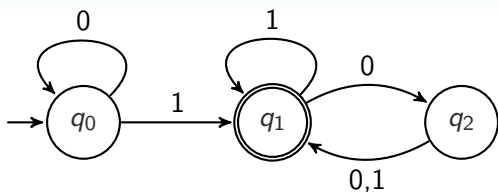


## Language of a DFA



For a DFA  $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ , the **language of  $\mathcal{A}$** ,  $L(\mathcal{A})$ , is the set of words from  $\Sigma^*$  which are accepted by  $\mathcal{A}$

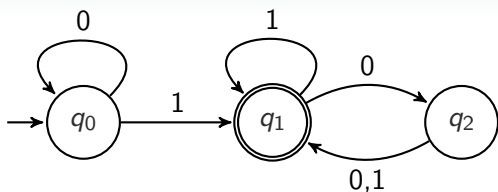
## Language of a DFA



$$L(\mathcal{A}) = \{1, 01, 11, 101, \dots\}$$

For a DFA  $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ , the **language of**  $\mathcal{A}$ ,  $L(\mathcal{A})$ , is the set of words from  $\Sigma^*$  which are accepted by  $\mathcal{A}$

## Language of a DFA



$$L(\mathcal{A}) = \{1, 01, 11, 101, \dots\}$$

For a DFA  $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ , the **language of**  $\mathcal{A}$ ,  $L(\mathcal{A})$ , is the set of words from  $\Sigma^*$  which are accepted by  $\mathcal{A}$

A language  $L \subseteq \Sigma^*$  is **regular** if there is some DFA  $\mathcal{A}$  such that  $L = L(\mathcal{A})$

# Language of a DFA: Formally

We extend the next state function to work on whole strings.

## Definition

**Base:**

$$\hat{\delta}(q, \epsilon) = q$$

**Induction:**

$$\hat{\delta}(q, aw) = \hat{\delta}(\delta(q, a), w)$$

## Example

For our running example,

$$\hat{\delta}(q_1, \epsilon) = q_1$$

$$\hat{\delta}(q_0, 010101000) = q_2$$

...

# Language of a DFA

## Definition

A DFA *accepts* a string  $w \in \Sigma^*$  iff  $\hat{\delta}(q_0, w) \in F$ .

## Definition

The *language* of a DFA  $A$  (written  $L(A)$ ) is the set of strings accepted by  $A$ .

$$L(A) = \left\{ w \in \Sigma^* \mid \hat{\delta}(q_0, w) \in F \right\}$$

# Outline

Motivation

Deterministic Finite Automata

**Non-deterministic Finite Automata**

NFAs vs. DFAs

$\epsilon$ -NFA

End Matter

# Non-determinism

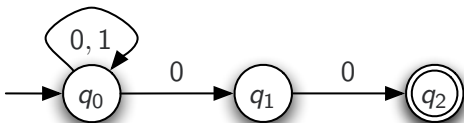
In a DFA, from each state

- 1 each input symbol leads to exactly one next state, and, therefore,
- 2 each input string leads to a unique state.

In a *non-deterministic finite automaton (NFA)*,

- 1 there can be several possible next states for each input symbol (or none), and
- 2 an input string can lead to several states (or no states).

The string is accepted by an NFA if *at least one* of these states is accepting.



## Essence of non-determinism

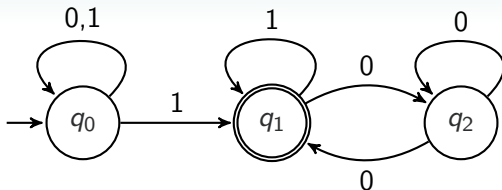
- Multiple alternative computations (often, huge numbers of them).
- Success if *any* succeed.

Different perspectives on this flavour of non-determinism:

- Massive parallel search.
- Search with “backtracking”
- Miraculously “guessing” the right option at each step.
- This is also known as *angelic* non-determinism in the literature.



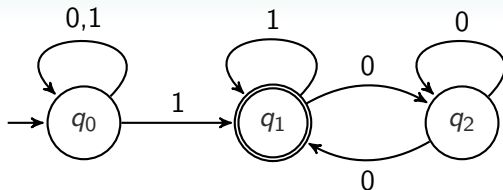
## Non-deterministic Finite Automata



A **non-deterministic finite automaton (NFA)** is a finite-state machine that has non-deterministic transitions.

More general than DFAs: A DFA is an NFA

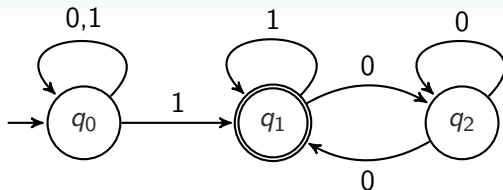
# Non-deterministic Finite Automata



Formally, a **non-deterministic finite automaton (NFA)** is a tuple  $(Q, \Sigma, \delta, q_0, F)$  where

- $Q$  is a finite set of states
- $\Sigma$  is the input alphabet
- $\delta : Q \times \Sigma \rightarrow 2^Q$  is the transition **relation**
- $q_0 \in Q$  is the start state
- $F \subseteq Q$  is the set of final/accepting states

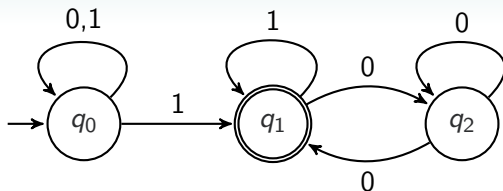
# Non-deterministic Finite Automata



Formally, a **non-deterministic finite automaton (NFA)** is a tuple  $(Q, \Sigma, \delta, q_0, F)$  where

- $Q$  is a finite set of states:  $Q = \{q_0, q_1, q_2\}$
- $\Sigma$  is the input alphabet
- $\delta : Q \times \Sigma \rightarrow 2^Q$  is the transition **relation**
- $q_0 \in Q$  is the start state
- $F \subseteq Q$  is the set of final/accepting states

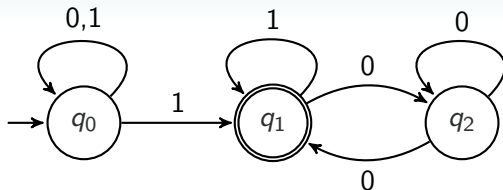
# Non-deterministic Finite Automata



Formally, a **non-deterministic finite automaton (NFA)** is a tuple  $(Q, \Sigma, \delta, q_0, F)$  where

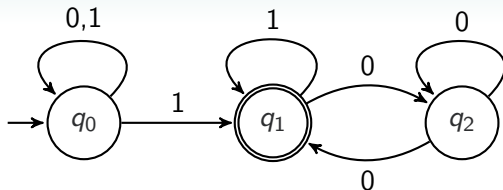
- $Q$  is a finite set of states:  $Q = \{q_0, q_1, q_2\}$
- $\Sigma$  is the input alphabet:  $\Sigma = \{0, 1\}$
- $\delta : Q \times \Sigma \rightarrow 2^Q$  is the transition **relation**
- $q_0 \in Q$  is the start state
- $F \subseteq Q$  is the set of final/accepting states

# Non-deterministic Finite Automata



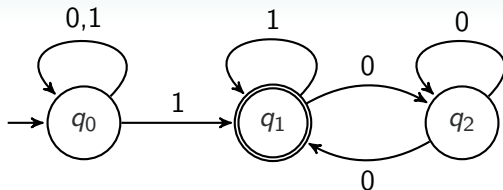
| $\delta$ | 0              | 1              |
|----------|----------------|----------------|
| $q_0$    | $\{q_0\}$      | $\{q_0, q_1\}$ |
| $q_1$    | $\{q_2\}$      | $\{q_1\}$      |
| $q_2$    | $\{q_1, q_2\}$ | $\emptyset$    |

## Non-deterministic Finite Automata



$$\delta = \left\{ \begin{array}{lll} (q_0, 0, q_0), & (q_0, 1, q_0), & (q_0, 1, q_1), \\ (q_2, 0, q_2), & (q_1, 0, q_2), & (q_1, 1, q_1), \\ & (q_2, 0, q_1) & \end{array} \right\}$$

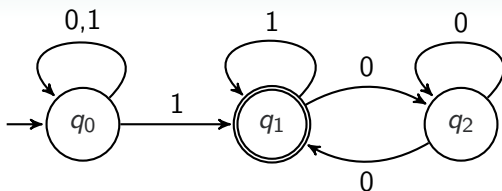
# Non-deterministic Finite Automata



Formally, a **non-deterministic finite automaton (NFA)** is a tuple  $(Q, \Sigma, \delta, q_0, F)$  where

- $Q$  is a finite set of states:  $Q = \{q_0, q_1, q_2\}$
- $\Sigma$  is the input alphabet:  $\Sigma = \{0, 1\}$
- $\delta : Q \times \Sigma \rightarrow 2^Q$  is the transition **relation**
- $q_0 \in Q$  is the start state
- $F \subseteq Q$  is the set of final/accepting states:  $F = \{q_1\}$

## Language of an NFA



An NFA accepts a sequence of symbols from  $\Sigma$  – i.e. elements of  $\Sigma^*$

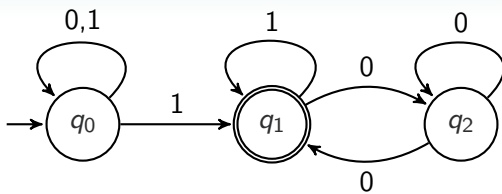
Informally: A word defines several runs in the NFA and the word is accepted if **at least one run** ends in a final state.

Note 1: Runs can end prematurely (these don't count)

Note 2: An NFA will always “choose wisely” (**Angelic** non-determinism)

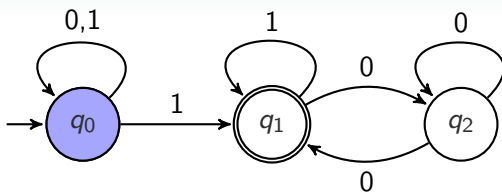


## Language of an NFA



$w$ : 1000

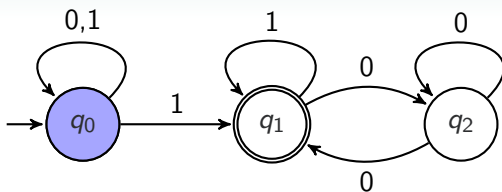
## Language of an NFA



$w$ : 1000

- Start in state  $q_0$

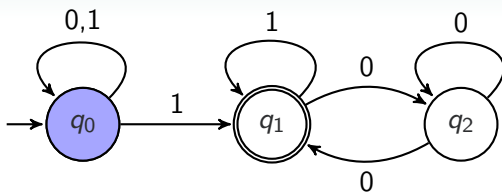
## Language of an NFA



$w$ : 1000

- Start in state  $q_0$
- Take the first symbol of  $w$

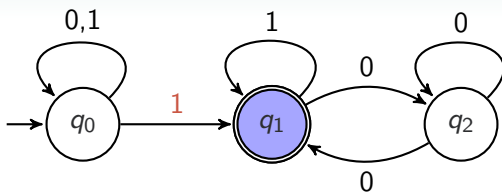
## Language of an NFA



$w$ : 1000

- Start in state  $q_0$
- Take the first symbol of  $w$
- Repeat until there are no symbols left or no transitions available:
  - Based on the current state and current input symbol, transition to any state determined by  $\delta$

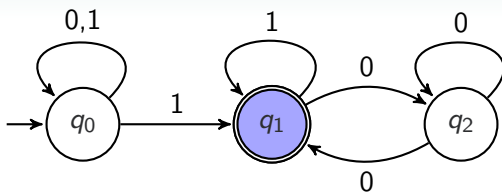
## Language of an NFA



$w$ : 1000

- Start in state  $q_0$
- Take the first symbol of  $w$
- Repeat until there are no symbols left or no transitions available:
  - Based on the current state and current input symbol, transition to any state determined by  $\delta$

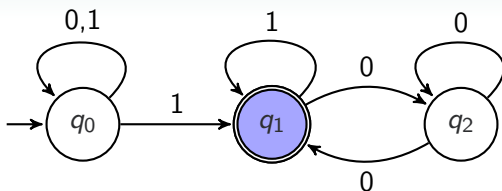
## Language of an NFA



$w$ : 1000

- Start in state  $q_0$
- Take the first symbol of  $w$
- Repeat until there are no symbols left or no transitions available:
  - Based on the current state and current input symbol, transition to any state determined by  $\delta$
  - Move to the next symbol in  $w$

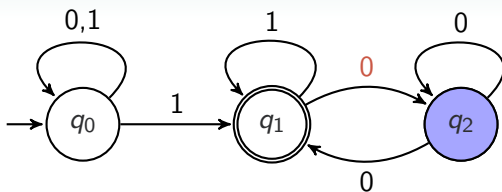
## Language of an NFA



$w$ : 1000

- Start in state  $q_0$
- Take the first symbol of  $w$
- Repeat until there are no symbols left or no transitions available:
  - Based on the current state and current input symbol, transition to any state determined by  $\delta$
  - Move to the next symbol in  $w$

## Language of an NFA

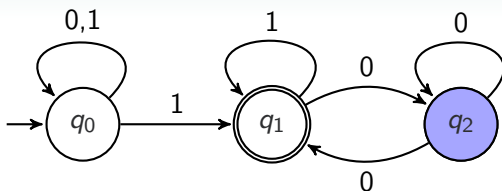


$w$ : 1000

- Start in state  $q_0$
- Take the first symbol of  $w$
- Repeat until there are no symbols left or no transitions available:
  - Based on the current state and current input symbol, transition to any state determined by  $\delta$
  - Move to the next symbol in  $w$



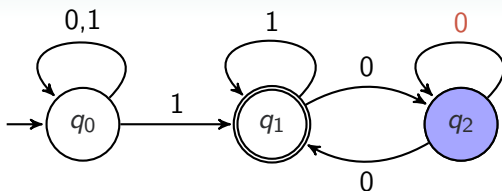
## Language of an NFA



$w$ : 1000

- Start in state  $q_0$
- Take the first symbol of  $w$
- Repeat until there are no symbols left or no transitions available:
  - Based on the current state and current input symbol, transition to any state determined by  $\delta$
  - Move to the next symbol in  $w$

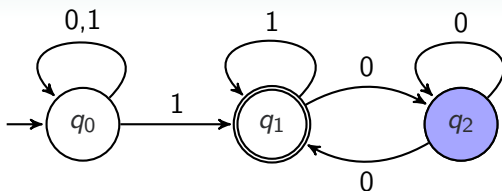
## Language of an NFA



$w$ : 1000

- Start in state  $q_0$
- Take the first symbol of  $w$
- Repeat until there are no symbols left or no transitions available:
  - Based on the current state and current input symbol, transition to any state determined by  $\delta$
  - Move to the next symbol in  $w$

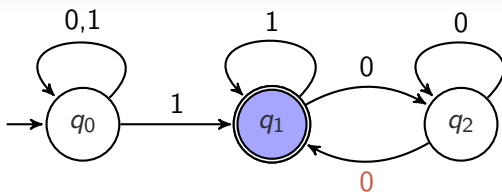
## Language of an NFA



$w$ : 1000

- Start in state  $q_0$
- Take the first symbol of  $w$
- Repeat until there are no symbols left or no transitions available:
  - Based on the current state and current input symbol, transition to any state determined by  $\delta$
  - Move to the next symbol in  $w$

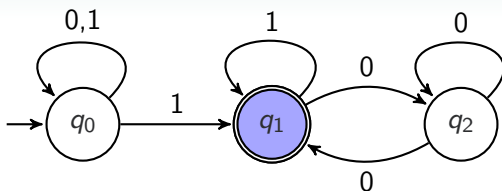
## Language of an NFA



$w$ : 1000

- Start in state  $q_0$
- Take the first symbol of  $w$
- Repeat until there are no symbols left or no transitions available:
  - Based on the current state and current input symbol, transition to any state determined by  $\delta$
  - Move to the next symbol in  $w$

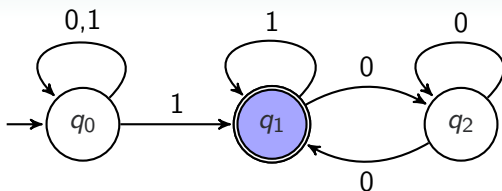
## Language of an NFA



$w$ : 1000

- Start in state  $q_0$
- Take the first symbol of  $w$
- Repeat until there are no symbols left or no transitions available:
  - Based on the current state and current input symbol, transition to any state determined by  $\delta$
  - Move to the next symbol in  $w$
- Accept if there are no symbols left and the process ends in a final state, otherwise reject.

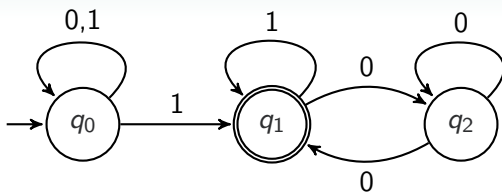
## Language of an NFA



$w$ : 1000 ✓

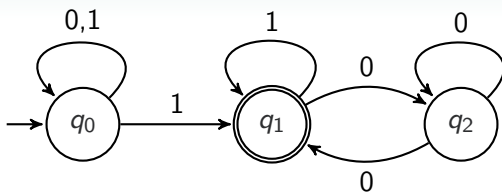
- Start in state  $q_0$
- Take the first symbol of  $w$
- Repeat until there are no symbols left or no transitions available:
  - Based on the current state and current input symbol, transition to any state determined by  $\delta$
  - Move to the next symbol in  $w$
- Accept if there are no symbols left and the process ends in a final state, otherwise reject.

## Language of an NFA



For an NFA  $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ , the **language of**  $\mathcal{A}$ ,  $L(\mathcal{A})$ , is the set of words from  $\Sigma^*$  which are accepted by  $\mathcal{A}$

## Language of an NFA



$$L(\mathcal{A}) = \{1, 01, 11, 100, 101, \dots\}$$

For an NFA  $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ , the **language of**  $\mathcal{A}$ ,  $L(\mathcal{A})$ , is the set of words from  $\Sigma^*$  which are accepted by  $\mathcal{A}$



# NFA Acceptance

## Definition (run of an NFA)

A *run* of an NFA  $A$  on a word  $w = a_1 \dots a_k$  is a sequence  $q_0 q_1 \dots q_k$  of states in  $Q$  such that

- $q_0$  is the initial state,
- for all  $i = 1 \dots k$ , we have  $q_i \in \delta(q_{i-1}, a_i)$ .

Such a run is *accepting* if the last state  $q_k \in F$ .

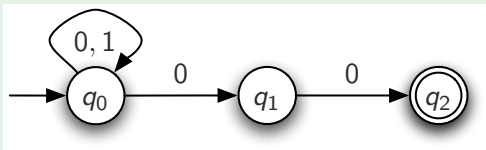
## Definition (Language of an NFA)

$$L(A) = \{ w \mid \text{There exists an accepting run of } A \text{ on } w \}$$

## NB

*It can be useful to talk about a run from a state  $q$  (other than the initial state).*

## Example



From the NFA  $A$  above, verify that the set of possible last states of runs of  $A$  on word 10100 is  $\{q_0, q_1, q_2\}$

Is  $10100 \in L(A)$ ? That is, from the start state, does the string lead to at least one final state?

# Outline

Motivation

Deterministic Finite Automata

Non-deterministic Finite Automata

NFAs vs. DFAs

$\epsilon$ -NFA

End Matter

## NFAs vs. DFAs

Amazingly, an NFA is no more powerful than a DFA (although it can sometimes be a lot smaller).

(With other kinds of automata, the non-deterministic versions are sometimes more powerful than the deterministic ones.)

### Theorem

*For every NFA  $N$ , there is a DFA  $D$  such that  $L(N) = L(D)$ .*

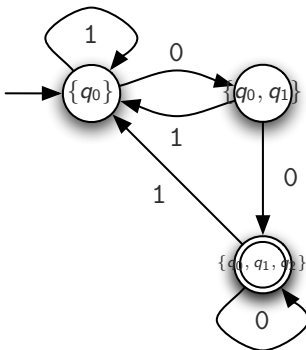
The proof relies on the *subset construction*.

## Subset Construction Example

The DFA produced by the subset construction *tracks* the set of states that the NFA can possibly be in (given the string read so far).

Hence, a state in the DFA = a set of NFA states.

Here is a tracking DFA for the above NFA:



The reasoning behind the construction was as follows:

... Suppose the original automaton was in one of the states in the set  $\{q_0, q_1\}$ , and we received a 0, what are all the states we can reach?

Clearly the answer is  $\{q_0, q_1, q_2\}$ .

Therefore there is an arrow in the DFA from the *DFA state*  $\{q_0, q_1\}$  to the DFA state  $\{q_0, q_1, q_2\}$ .

## Subset Construction

(Back to the proof of NFAs being no more powerful than DFAs.)

Given an NFA  $N = (Q, \Sigma, q_0, \delta_N, F_N)$ , construct a DFA

$$D = (2^Q, \Sigma, \{q_0\}, \delta_D, F_D)$$

such that  $L(D) = L(N)$ .

The key is in the definition of  $\delta_D$  and  $F_D$ .

$$\delta_D(S, a) = \bigcup_{q \in S} \delta_N(q, a) \text{ , for each } S \subseteq Q.$$

The final states of  $D$  are the subsets that contain *at least one* final state of  $N$ :

$$F_D = \{ S \subseteq Q \mid S \cap F_N \neq \emptyset \} \text{ .}$$

# Correctness of the subset construction

## Theorem

*If  $N$  is an NFA and  $D$  is the DFA obtained by applying subset construction to  $N$  then  $L(N) = L(D)$ .*

But it's actually easier to prove a stronger claim:

## Lemma

*If  $N$  is an NFA and  $D$  is the DFA obtained by applying subset construction to  $N$  then  $\hat{\delta}_D(S, w)$  is the set of all states  $q'$  such that there exists a  $q \in S$  and a run  $qq_1 \dots q'$  of  $N$  on  $w$ , with last state  $q'$ .*

(I.e., the state you get to by running  $D$  on  $w$  is the same as the set of states you get to by running  $N$  on  $w$ .)



# Correctness of the subset construction

## Theorem

*If  $N$  is an NFA and  $D$  is the DFA obtained by applying subset construction to  $N$  then  $L(N) = L(D)$ .*

But it's actually easier to prove a stronger claim:

## Lemma

*If  $N$  is an NFA and  $D$  is the DFA obtained by applying subset construction to  $N$  then  $\hat{\delta}_D(S, w)$  is the set of all states  $q'$  such that there exists a  $q \in S$  and a run  $qq_1 \dots q'$  of  $N$  on  $w$ , with last state  $q'$ .*

(I.e., the state you get to by running  $D$  on  $w$  is the same as the set of states you get to by running  $N$  on  $w$ .)

## Correctness of the subset construction

Proof by induction on the length of  $w$ . Write  $Last(q, w)$  for the set of last states of runs of  $N$  on  $w$ , starting from  $q$ . We need to show  $\hat{\delta}_D(S, w) = \bigcup_{q \in S} Last(q, w)$ .

Reminder: **Base:**  $\hat{\delta}_D(S, \epsilon) = S$ ,

**Induction:**  $\hat{\delta}_D(S, aw) = \hat{\delta}_D(\delta_D(S, a), w) = \hat{\delta}_D(\bigcup_{q \in S} \delta_N(q, a), w)$

**Base Case:**  $|w| = 0$ , i.e. ,  $w = \epsilon$ . Then the only run of  $N$  from  $q$  on  $w$  is  $q$ , so  $Last(q, w) = \{q\}$ . But for all sets  $S$ ,  $\hat{\delta}_D(S, \epsilon) = S$  by definition, so  $\hat{\delta}_D(S, w) = \bigcup_{q \in S} \{q\} = \bigcup_{q \in S} Last(q, w)$ .

## Correctness of the subset construction (ctd)

**Inductive case:** Suppose that  $\hat{\delta}_D(S', w) = \bigcup_{q \in S'} \text{Last}(q, w)$  for all sets  $S'$ . We show that, for  $a \in \Sigma$ , we have

$$\hat{\delta}_D(S, aw) = \bigcup_{q \in S} \text{Last}(q, aw).$$

From the definition of a run,  $\text{Last}(q, aw) = \bigcup_{q' \in \delta_N(q, a)} \text{Last}(q', w)$ ; so we have

$$\begin{aligned} \hat{\delta}_D(S, aw) &= \hat{\delta}_D(\bigcup_{q \in S} \delta_N(q, a), w) \\ &= \hat{\delta}_D(\bigcup_{q \in S} \bigcup_{q' \in \delta_N(q, a)} \{q'\}, w) \\ &= \bigcup_{q \in S} \bigcup_{q' \in \delta_N(q, a)} \text{Last}(q', w) \\ &= \bigcup_{q \in S} \text{Last}(q, aw). \end{aligned}$$

# Equivalence of NFAs and DFAs

## Theorem

*A language  $L$  is  $L(N)$  for some NFA  $N$  iff it is  $L(D)$  for some DFA  $D$ .*

## Proof.

The subset construction was the hard part of this: Given any NFA, a DFA accepting the same language can be constructed. So, whenever a language is accepted by an NFA, it is also accepted by some DFA.

The other direction of the proof is obvious. Every DFA is essentially an NFA. The only change is that  $\delta_N(q, a) = \{\delta_D(q, a)\}$ . All other aspects of the FA remain the same, including  $F$ .  $\square$

## Subset Construction Efficiency

In the worst case, the subset construction generates  $2^{|Q|}$  states.

In practice, it is often unnecessary to generate them all, since few are actually reachable by any input string.

A more efficient procedure: Generate the subset *on the fly*. Create them as you trace through the NFA. That way, you only generate the *accessible* subsets.

### Example

Notice that in our subset construction example the DFA states for the sets  $\{q_0, q_2\}$  and the  $\emptyset$  are absent in the final DFA obtained.

Clearly, we used the efficient procedure to compute it!

# Outline

Motivation

Deterministic Finite Automata

Non-deterministic Finite Automata

NFAs vs. DFAs

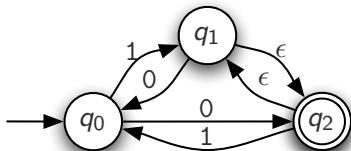
$\epsilon$ -NFA

End Matter

## $\epsilon$ -NFA

**Idea:** Allow state changes that don't consume input symbols (“silent moves”).

Choice of whether to take  $\epsilon$ -transition is non-deterministic.



The input “001” is accepted. A possible run is  $q_0 q_2 q_1 q_0 q_1 q_2$ .

## Definition of $\epsilon$ -NFA

The only difference in the mathematical definition is in  $\delta$ :

$$\delta : Q \times (\Sigma \cup \{\epsilon\}) \longrightarrow 2^Q$$

### NB

*Sipser's notation sometimes varies from ours.*

|                  |                         |
|------------------|-------------------------|
| [Sipser06]       | COMP4141                |
| <i>NFA</i>       | $\epsilon$ - <i>NFA</i> |
| $\mathcal{P}(Q)$ | $2^Q$                   |



# Acceptance by an $\epsilon$ -NFA

## Definition (run of an $\epsilon$ -NFA)

A *run* of an  $\epsilon$ -NFA  $A$  on a word  $w$  is a sequence  $q_0 q_1 \dots q_k$  of states in  $Q$  such that we can write  $w = a_1 \dots a_k$  with each  $a_i \in \Sigma \cup \{\epsilon\}$  and

- $q_0$  is the initial state,
- for all  $i = 1 \dots k$ , we have  $q_i \in \delta(q_{i-1}, a_i)$ .

Such a run is *accepting* if the last state  $q_k \in F$ .

This is almost the same as the NFA definition, but note that since  $\epsilon u = u \epsilon = u$ , for  $w = 01$ , we can write  $w = \epsilon \epsilon 0 \epsilon 1 \epsilon$ , and there are (infinitely) many other ways to insert  $\epsilon$  steps.

## Definition (Language of an $\epsilon$ -NFA)

$$L(A) = \{ w \mid \text{There exists an accepting run of } A \text{ on } w \}$$

## $\epsilon$ -closure

Preliminary: the  $\epsilon$ -closure  $E(q)$  of state  $q$  is the set of all states reachable from  $q$  by silent moves.

$$E(q_0) = \{q_0\}$$

$$E(q_1) = \{q_1, q_2\}$$

$$E(q_2) = \{q_1, q_2\}$$

## Definition of $E$

[ $S$  is the *least* set satisfying property  $P$  if  $P(S)$ , and whenever  $P(S')$  holds,  $S \subseteq S'$ .]

### Definition

$E(q)$  is the least set satisfying:

- $q \in E(q)$
- $\delta(s, \epsilon) \subseteq E(q)$  for all  $s \in E(q)$

Also,  $E(S) = \bigcup \{ E(q) \mid q \in S \}$ .

(“Extend to sets” — this is overloading)

# Converting an $\epsilon$ -NFA to a DFA

## Theorem

*For every  $\epsilon$ -NFA  $N$ , there exists an DFA  $D$  such that  $L(D) = L(N)$ .*

## Proof.

The proof uses a modified subset construction, where every subset is  $\epsilon$ -closed.

Given  $N = (Q, \Sigma, q_0, \delta_N, F_N)$ , we can define  $D = (2^Q, \Sigma, E(q_0), \delta_D, F_D)$  as follows:

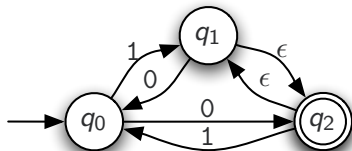
$$F_D = \{ S \subseteq Q \mid S \cap F_N \neq \emptyset \}$$

$$\delta_D(S, a) = E\left(\bigcup_{s \in S} \delta_N(s, a)\right)$$

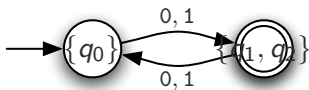


Note:  $\delta_D$  has no  $\epsilon$  transitions!

## $\epsilon$ -elimination example



converts to the DFA:



Theorem:  $L(N) = L(D)$ .

The proof is as in equivalence of NFAs and DFAs.

# Summary

DFAs, NFAs, and  $\epsilon$ -NFAs all have equal expressive power.

I.e., given a language  $L$ , if one kind of FA accepts  $L$ , then there are acceptors for  $L$  in the other kinds of FA, too.

# Outline

Motivation

Deterministic Finite Automata

Non-deterministic Finite Automata

NFAs vs. DFAs

$\epsilon$ -NFA

End Matter



## Weekly Feedback

I would appreciate any comments/suggestions/requests you have on this week's lectures (and tutorials).



<https://forms.office.com/r/g3LgPASave>